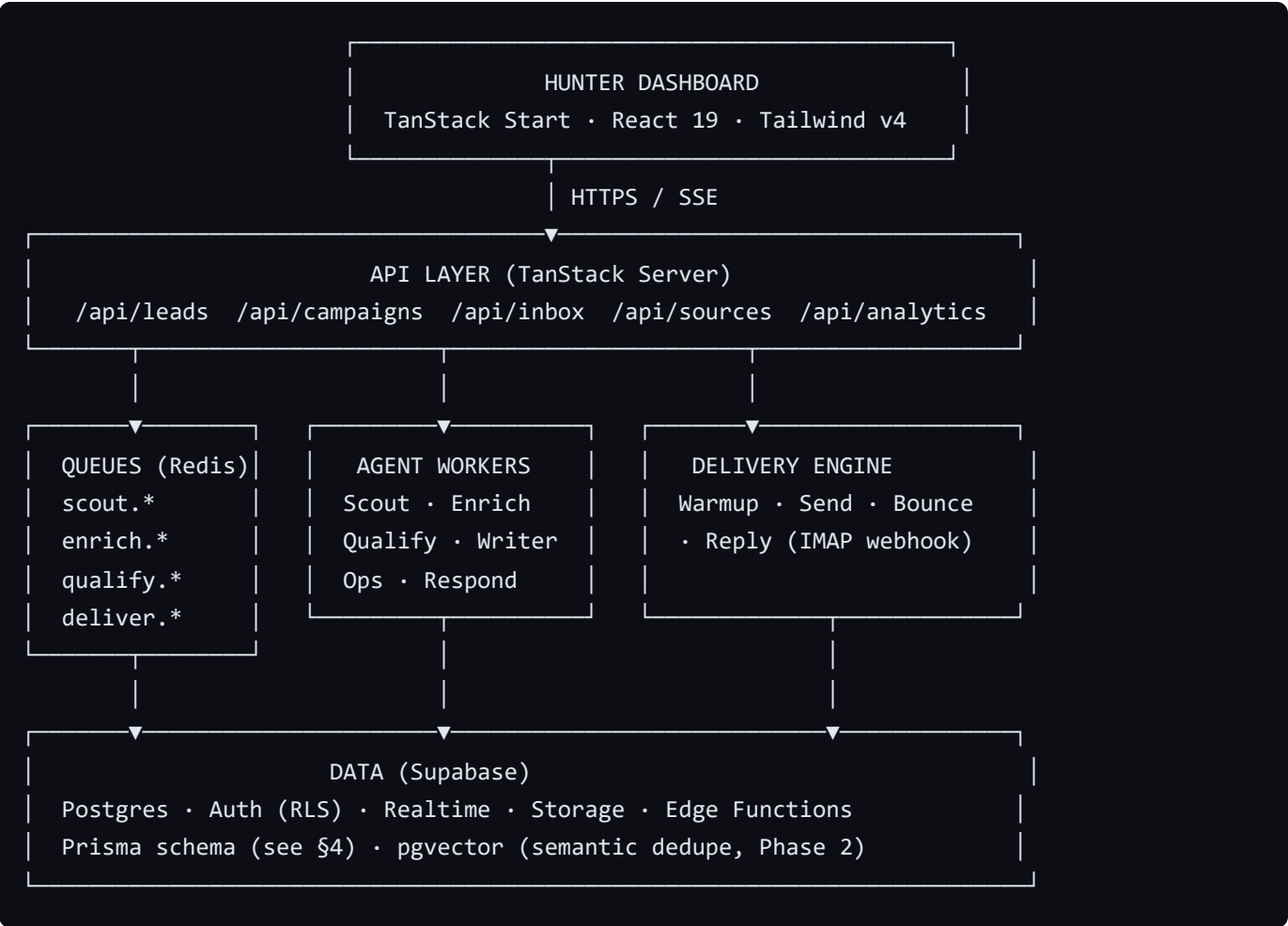


Signhify Hunter — Technical Requirements Document (TRD)

Codename: HUNTER · **Version:** 1.0 · **Date:** 07 Aug 2026 Companion docs: `01_PRD.md` , `03_IMPLEMENTATION_PLAN.md` , `04_FRONTEND_PRD_TRD.md` , `05_BACKEND_PRD_TRD.md`

1. Architecture Overview

Monorepo, TypeScript everywhere, Supabase-first, job-queue driven agent pipeline.

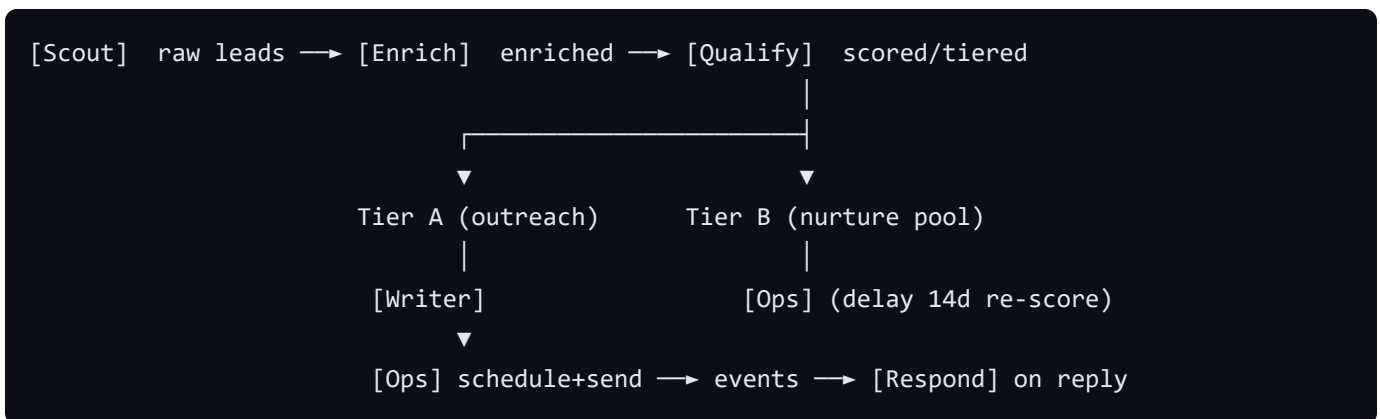


1.1 Technology Stack (mirrors Signhify studio standard)

Layer	Choice	Why
Language	TypeScript (strict)	Studio standard, one language end-to-end

Layer	Choice	Why
Frontend	TanStack Start, React 19, Tailwind v4, shadcn/ui, Framer Motion	Same as signhify.dpdns.org / signhify-ai-web
Backend	TanStack Server routes + Node workers	Shared types, edge-ready
DB	Supabase Postgres + Prisma	Auth/RLS/Realtime for free; Prisma for typed schema
Queue	Upstash Redis + ack-aware worker loop	Serverless-friendly, cheap, no infra
Email	Resend (primary) + SMTP pool (secondary domains)	Studio already uses Resend (TuitionTrack)
Inbound	Resend webhooks → inbox; IMAP polling fallback	Reliability
AI	OpenAI (GPT-4.1/4o mini) + Anthropic (Claude Sonnet)	Qualify/Write/Respond; cost-tiered (mini for bulk)
Scraping	Firecrawl API + plain fetch for public JSON feeds	Studio skill; ToS-aware
Verification	Self-built (syntax→MX→SMTP) + Hunter.io/Apollo API hybrid	Cost control
Billing	Stripe (subscription + usage metering)	Studio standard
Deploy	Vercel (web/api) + Cloudflare Workers (cron jobs)	Studio standard
Secrets	Supabase secrets + <code>.env</code> (never committed)	—
Observability	OpenTelemetry → Axiom/Sentry + structured logs	Cheap, searchable

1.2 Agent Pipeline (event-driven)



Each stage is a queue; each job idempotent; each stage stores an `Event` row (audit trail).

2. System Requirements

2.1 Functional components

Component	Requirement
Source adapters	One adapter per channel; config-driven; per-source rate limits; schema-versioned output
Dedupe	Domain-keyed <code>leads.organization_domain</code> unique per workspace; fuzzy via pgvector (Phase 2)
Verification pipeline	syntax → disposable-domain check → MX → SMTP (250/550/retry) → verdict (verified/risky/failed)
Scoring	LLM with structured output (JSON schema); deterministic pre-filters (country, industry, recency) run before LLM to save tokens
Writer	Per-lead JSON: <code>{subject, body}</code> ; template system with <code>{{lead.field}}</code> slots + LLM personalization; 120-word cap enforced
Delivery	Cron-driven scheduler picks due sends; respects limits/windows; at-least-once with idempotency key
Warmup	Daily script: sends to warmup network (Seedleads/self-hosted pool); ramps 5→30/day over 14 days
Inbound	Webhook → parse (message-id, from, subject, body) → classify → notify via Realtime to dashboard
Suppression	Global list checked at send time (postgres lookup, cached in Redis); unsubscribes write immediately

2.2 Non-functional

- **Latency:** dashboard p95 < 2s; agent job p95 < 30s per lead (except SMTP verify ~5s).
- **Throughput:** V1: 50k leads, 1M events/month; design headroom 10×
- **Reliability:** at-least-once delivery with idempotent handlers; DLQ with replay; 99.5% target.
- **Security:** RLS everywhere; service-role key only in workers; approval audit log; no PII in logs; secrets rotated.
- **Compliance:** `list-unsubscribe` + unsubscribe page + physical address; DSAR endpoint; retention policy (leads deleted 90d after last touch unless replied); suppression is permanent.

3. API Specification (internal + public surface)

3.1 REST API (public surface, session auth)

Method	Path	Purpose
GET	<code>/api/leads?source=&tier=&q=</code>	List/search leads (paginated)

Method	Path	Purpose
POST	/api/leads/{id}/tier	Manual re-tier
GET	/api/leads/{id}	Detail w/ enrichment + events
GET/POST	/api/sources	Source configs CRUD
GET/POST	/api/campaigns	Campaigns CRUD
POST	/api/campaigns/{id}/run	Start/stop campaign
GET	/api/campaigns/{id}/analytics	Funnel stats
GET	/api/inbox	Threads, unread count
POST	/api/inbox/{threadId}/reply	Approve & send reply (HITL)
POST	/api/inbox/{threadId}/book	Create meeting link + move to won
GET	/api/analytics/dashboard	Global KPIs
GET/POST	/api/settings/domains	Sending domains + DNS check
GET	/api/usage	Metered usage (billing)
POST	/api/webhooks/resend	Inbound email webhook (public)
GET	/api/compliance/unsubscribe?id=&list=	One-click unsubscribe (public, no auth)
POST	/api/compliance/dsar	Erasure/data request (public)

3.2 Agent Worker API (internal)

Workers consume Redis streams; all functions idempotent:

- `scout.run(sourceId, cursor) → lead[]`
- `enrich.verify(leadId) → verification`
- `enrich.company(leadId) → enrichment`
- `qualify.score(leadId) → {score, tier, reasons[]}`
- `writer.compose(leadId, campaignId, stepId) → message[]`
- `ops.send(messageId) → status`
- `ops.warmup(domainId) → count`
- `respond.classify(threadId) → {category, suggestion}`

4. Data Model (Prisma)

4.1 Core tables

```

model Workspace { id, name, slug, created_at, Users[], Campaigns[], Domains[], Settings? }
model User { id, email, workspace_id?, role, approvals[] }

model Lead {
  id, workspace_id, organization_name, organization_domain @unique(workspace)
  website, industry, country, size_range, tech_stack String[] // jsonb
  contact_name, contact_role, contact_linkedin, contact_email
  source_url, source_channel, source_raw Json
  email_verification EmailVerdict // verified|risky|failed|unknown
  score Int?, tier Tier? // A|B|C
  score_reason String?, status LeadStatus // new|qualified|contacted|replied|meeting|unsubscribe
  created_at, updated_at, Events[], Messages[], CampaignLeads[]
}

model SourceConfig {
  id, workspace_id, channel, name, config Json // keywords, filters, geo, limits
  enabled, last_run_at, cursor Json, rate_limit_per_hour, created_at
}

model Campaign {
  id, workspace_id, name, status // draft|running|paused|stopped
  audience Json, Steps[], CampaignLeads[], created_at
}

model CampaignStep { id, campaign_id, channel, order, delay_days, subject_template, body_template }

model CampaignLead { id, campaign_id, lead_id, step_index, next_send_at, status }

model Message {
  id, workspace_id, lead_id, campaign_id, channel, direction // out|in
  subject, body, provider_message_id @unique, status // queued|sent|bounced|delivered|opened|replied
  sent_at, Events[]
}

model Event { id, workspace_id, lead_id?, message_id?, type, payload Json, created_at } // full

model Thread { id, workspace_id, lead_id, subject, status // new|needs_reply|replied|booked
  category // positive|question|negative|unsubscribe|other
  suggestion Json?, Reply[], created_at, updated_at }

model Reply { id, thread_id, direction, body, approved_by, status // draft|approved|sent|rejected }

model SendingDomain { id, workspace_id, domain, provider, dkim_status, spf_status, dmarc_status
  warmup_level Int, daily_limit Int, sent_today Int, blacklisted Bool, created_at }

model SuppressionList { id, workspace_id, email @unique, reason, source, created_at } // permanent

```

```
model Booking { id, workspace_id, lead_id, thread_id, meeting_url, scheduled_at, status }
```

4.2 Indexes & RLS

- Indexes: `Lead(workspace_id, tier)`, `Lead(organization_domain)`, `Message(provider_message_id)`, `Event(lead_id, created_at)`, `CampaignLead(campaign_id, next_send_at)`.
- RLS: every table `workspace_id = auth.uid()→workspace_id`; public rows only for the two compliance endpoints (unsubscribe/DSAR) via edge function + token, never table grants.
- Retention job (cron, daily): delete leads untouched 90d; write audit event first.

5. AI Agent Specifications

5.1 Shared conventions

- Structured outputs: all LLM calls use JSON-schema-constrained output; parse failures retried once, then `fallback` tier.
- Model routing: bulk classify/score → `gpt-4o-mini`; final Writer & Respond → Claude Sonnet 4 (studio uses Anthropic).
- Cost budget: < \$0.02/lead end-to-end (scoring + writing); budget cap per campaign with alarm.
- Every agent writes an `Event` row (inputs hash, model, tokens, latency).

5.2 Agent prompts (summary contracts)

Agent	Input	Output schema (key fields)
Scout	source config + cursor	<code>leads[] {name, domain, url, contact?, context, source_url}</code>
Qualify	enriched lead + ICP rules	<code>{score: 0-100, tier, reasons: string[], signal_summary}</code>
Writer	lead + step template	<code>{subject ≤60 chars, body ≤120 words, cta}</code>
Respond	thread + lead context	<code>{category, intent, suggested_reply ≤80 words, needs_human: bool}</code>

ICP rules (v1, editable in settings): - Sells to: founders/startups/SMBs needing software, agencies needing delivery partners, scale-ups with AI/product gaps. - Signals: mentions of "agency/developer/hire/build app/automation/website", new funding, fresh product launch, job posts for devs, poor/no existing digital presence. - Budget proxy: company size 2–500, industry any, geography: US/UK/EU/AU/IN/Middle East preferred, no schools/charities (default off).

6. Integrations

Integration	Use	Direction
Resend	Outbound email, inbound webhooks, open/click events	in/out
SMTP (custom)	Extra sending domains	out
Hunter.io / Apollo	Enrichment + verification supplement	in
Firecrawl	Directory/website scraping	in
GitHub API	Repo/org signals	in
HN Algolia / Reddit JSON / PH API	Public feeds	in
Upwork (rss)	Project listings	in
SerpAPI / Brave	SERP sourcing	in
Stripe	Plans + usage metering	out
Cal.com/Calendly	Meeting booking	out
CSV export + (Phase 2) HubSpot API	CRM export	out

7. Environments & Tooling

Item	Spec
Repo	<code>signhify/hunter</code> monorepo (pnpm workspaces)
Branching	<code>main</code> (protected) + feature branches; PR review; semantic commits
CI	GitHub Actions: typecheck, lint, unit tests, build; Playwright smoke on PR
Test	Vitest (unit), MSW (API), Playwright (E2E), fixture-based agent tests w/ mock LLM
Local	<code>docker compose</code> (postgres+redis), supabase CLI, hot reload
Observability	OpenTelemetry → Axiom, Sentry errors, health checks on <code>/api/health</code>
Secrets	1Password vault + CI secret store; <code>.env</code> gitignored (see <code>.env.example</code>)